Lecture 15

PyTorch Continued & Backpropagation

Backpropagation – computing gradients easily and efficiently!

EECS 189/289, Fall 2025 @ UC Berkeley
Joseph E. Gonzalez and Narges Norouzi

Join at slido.com
#3940975

Implement Gradient Descent with Optimizer

3940975

In PyTorch you implement **your own gradient descent loop** but you can use built in update rules (e.g., Adam) called **optimizers**.

```
loader = DataLoader(tr_data, batch_size=32, shuffle=True)
optimizer = torch.optim.Adam(model.parameters(),eta)

for epoch in range(nepochs):
    for (x, t) in loader:
        optimizer.zero_grad()
        loss = loss_fn(model(x), t)
        loss.backward()
        optimizer.step()
```

Minibatch SGD DataLoader

Adam optimizer

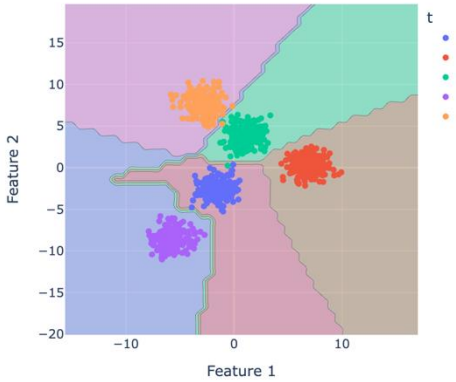
Epoch is one pass over data

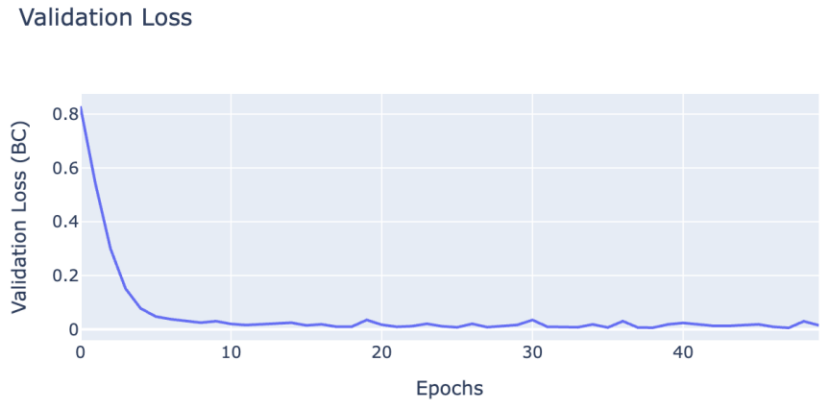
Load one minibatch at a time

Compute the gradient of the loss

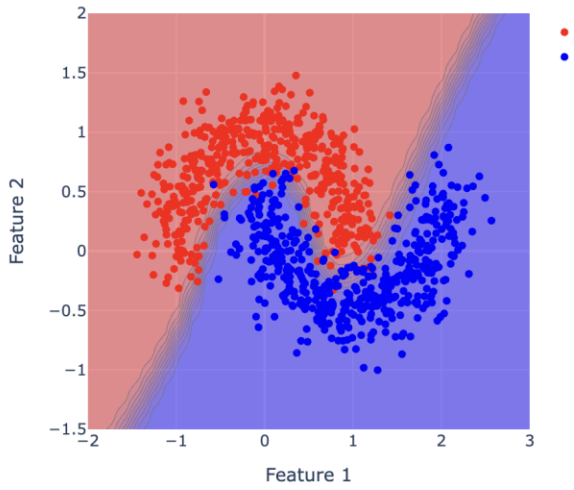
Use optimizer to update all the parameters (1-step)

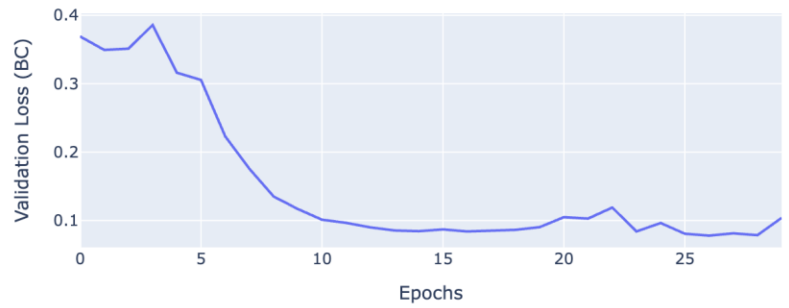
Demo

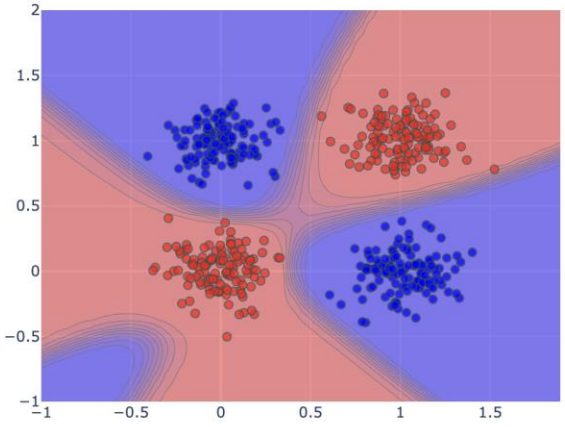




3940975







Recap: Stochastic **Gradient Descent**

Iteratively refine the weights by **computing the gradient** of the loss on a small sample of data.

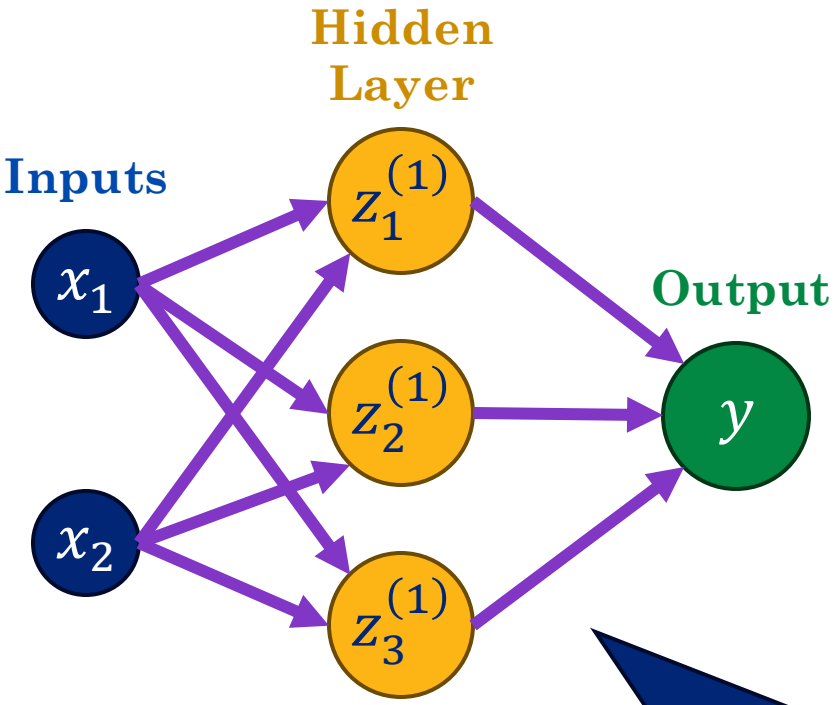
$\mathbf{w}^{(0)} = \text{Initialize}()$
for τ in range$(1, \tau_{\max})$:
    $\mathcal{B} = \text{RandBatch}(0, N, \text{size} = B)$
    $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \text{Step}\left(\nabla E_{\mathcal{B}}\left(\mathbf{w}^{(\tau-1)}\right), \tau, \beta, \eta\right)$
    if $\text{Converged}\left(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon\right)$: terminate

- This algorithm is central to all recent advances in AI
- Today we will learn how to compute $\nabla E_{\mathcal{B}}$ **efficiently** and **automatically**

Simple Running Example

3940975

Consider the following single hidden layer classification network



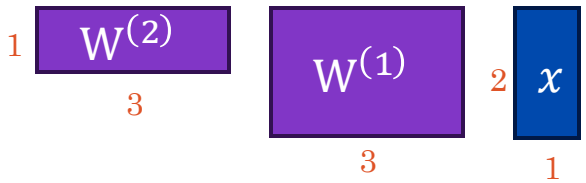
$$z_i^{(1)} = \sigma\left(\sum_{j=1}^{2} w_{ij}^{(1)} x_j\right)$$

Sigmoid Activation Function

$$\sigma(t) = \frac{1}{1+e^{-t}}$$

$$y(\mathrm{x}, \mathrm{w}) = \sigma\left(\sum_{i=1}^{3} w_{1i}^{(2)} z_i^{(1)}\right) = \sigma\left(\sum_{i=1}^{3} w_{1i}^{(2)} \sigma\left(\underbrace{\sum_{j=1}^{2} w_{ij}^{(1)} x_j}_{\text{Substituting } z_i^{(1)}}\right)\right)$$

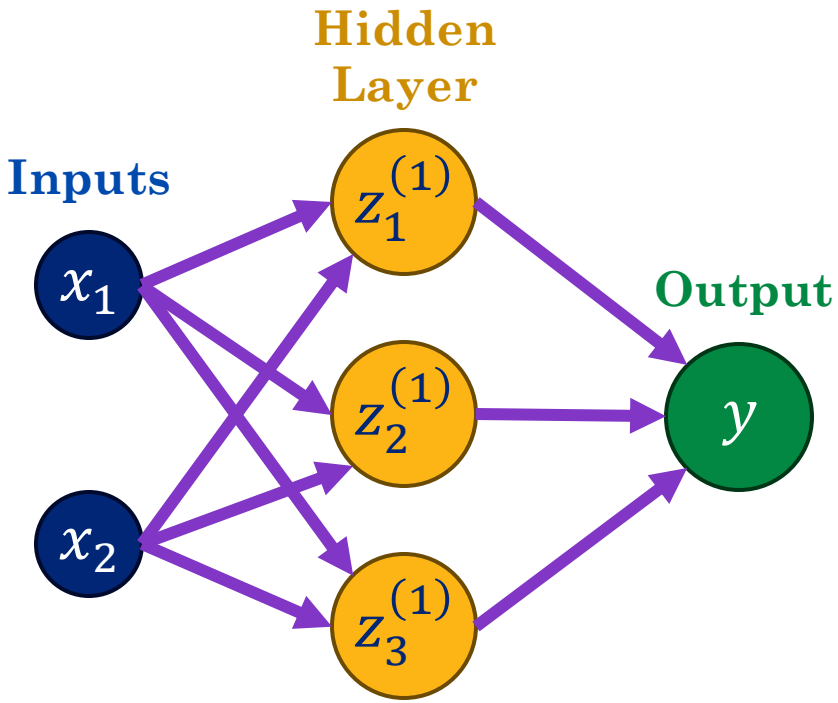
$$p(t=1 \mid x, w) = y(\mathrm{x}, \mathrm{w}) = \underbrace{\sigma\left(\mathrm{W}^{(2)} \sigma\left(\mathrm{W}^{(1)} x\right)\right)}_{\text{Matrix Notation}}$$



No **bias at each layer** in this example.

Defining the Error Function

Computing the negative log likelihood of the data $\mathcal{D}$:



$$p(t = 1 \mid x, w) = y(\mathrm{x}, \mathrm{w}) = \sigma\left(\mathrm{W}^{(2)}\sigma\left(\mathrm{W}^{(1)}x\right)\right)$$

$$E(\mathrm{w}; \mathcal{D}) = -\ln\left(\underbrace{\prod_{d=1}^{D} y(\mathrm{x}_d, \mathrm{w})^{t_d}\left(1 - y(x_d, w)\right)^{(1-t_d)}}_{\text{Likelihood of the data.}}\right)$$

$$E(\mathrm{w}; \mathcal{D}) = \underbrace{-\sum_{d=1}^{D} t_d \ln(y(\mathrm{x}, \mathrm{w})) + (1 - t_d)\ln(1 - y(\mathrm{x}, \mathrm{w}))}_{\text{Algebra!}}$$

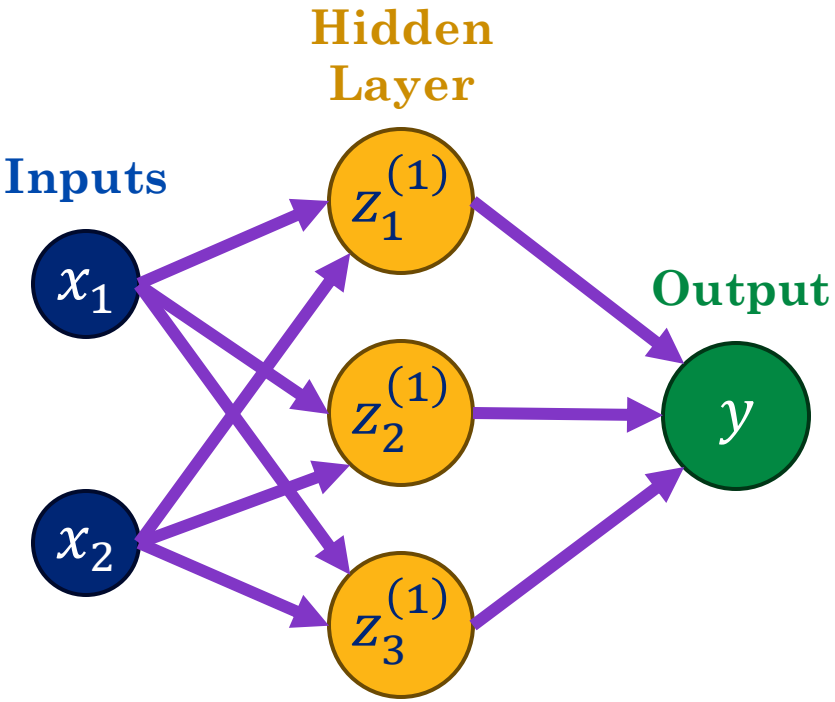
Sigmoid Activation Function

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

Computing the Gradient

3940975

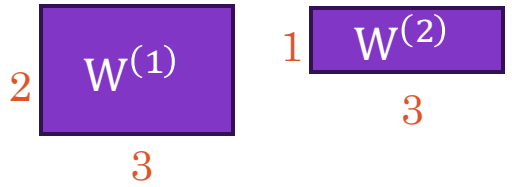
Computing the gradient ∇_w of the negative log likelihood with respect to w:



$$p(t = 1 \mid x, w) = y(\mathrm{x}, \mathrm{w}) = \sigma\left(\mathrm{W}^{(2)}\sigma\left(\mathrm{W}^{(1)}x\right)\right)$$

$$E(\mathrm{w}; \mathcal{D}) = -\sum_{d=1}^{D} t_d \ln(y(\mathrm{x}, \mathrm{w})) + (1 - t_d)\ln(1 - y(\mathrm{x}, \mathrm{w}))$$

What is the shape of the gradient?
Recall $\mathrm{w} = \{\mathrm{W}^{(1)}, \mathrm{W}^{(2)}\}$



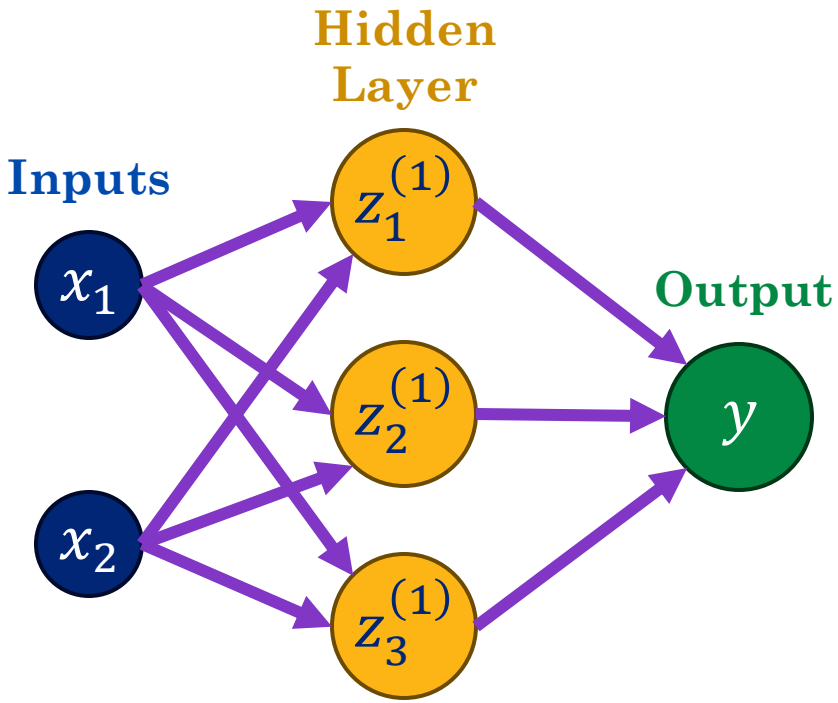
Sigmoid Activation Function

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

What is the shape of the gradient?

Computing the Gradient

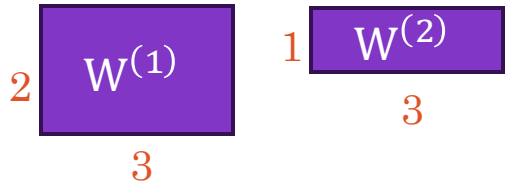
Computing the gradient ∇_w of the negative log likelihood with respect to w:



$$p(t = 1 \mid x, w) = y(\mathrm{x}, \mathrm{w}) = \sigma\left(\mathrm{W}^{(2)}\sigma\left(\mathrm{W}^{(1)}x\right)\right)$$

$$E(\mathrm{w}; \mathcal{D}) = -\sum_{d=1}^{D} t_d \ln(y(\mathrm{x}, \mathrm{w})) + (1 - t_d)\ln(1 - y(\mathrm{x}, \mathrm{w}))$$

What is the shape of the gradient?
Recall $\mathrm{w} = \{\mathrm{W}^{(1)}, \mathrm{W}^{(2)}\}$



The gradient ∇_w is the **vector** of **partial derivatives** for all the entries in $\mathrm{W}^{(1)}$ and $\mathrm{W}^{(2)}$.

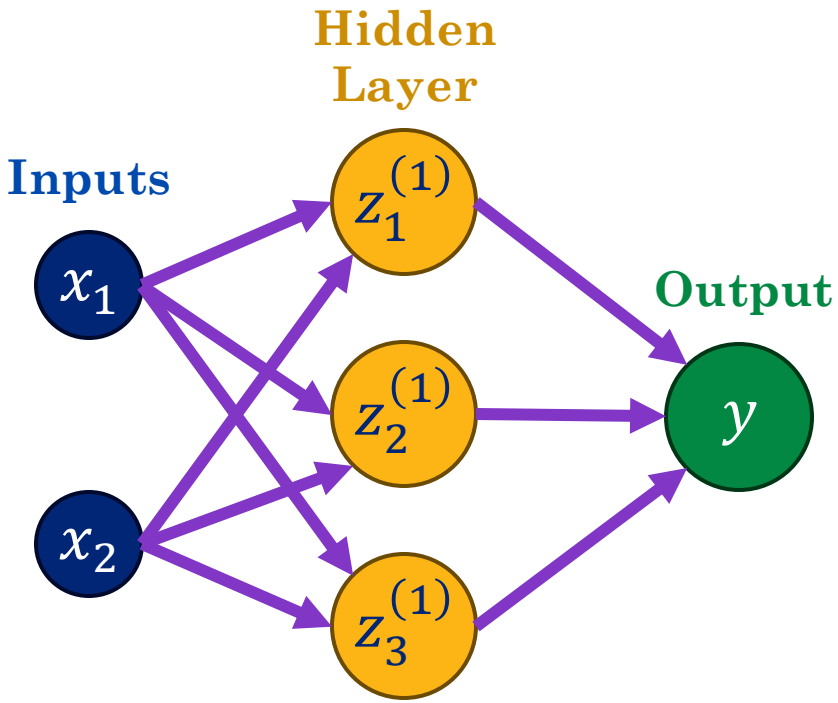
Here $\nabla_w E(\mathrm{w}; \mathcal{D}) \in \mathbb{R}^{(2*3+3)} = \mathbb{R}^9$ (9 edge in network graph)

Sigmoid Activation Function

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

Computing the Gradient

Using the chain rule to compute the partial derivative with respect to w_i



Sigmoid Activation Function

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

$$p(t = 1 \mid x, w) = y(\mathrm{x}, \mathrm{w}) = \sigma\left(\mathrm{W}^{(2)}\sigma\left(\mathrm{W}^{(1)}x\right)\right)$$

$$E(\mathrm{w}; \mathcal{D}) = -\sum_{d=1}^{D} t_d \ln(y(\mathrm{x}, \mathrm{w})) + (1 - t_d)\ln(1 - y(\mathrm{x}, \mathrm{w}))$$

Recall that $\frac{\partial}{\partial q}\ln(\mathrm{q}) = \frac{1}{q}$ then with the **chain rule**:

$$\frac{\partial}{\partial w_i}E(\mathrm{w}; \mathcal{D}) = -\sum_{d=1}^{D} t_d \left(\frac{1}{y(\mathrm{x}, \mathrm{w})}\right)\frac{\partial y(\mathrm{x}, \mathrm{w})}{\partial w_i} + (1 - t_d)\left(\frac{1}{1 - y(\mathrm{x}, \mathrm{w})}\right)\frac{\partial y(\mathrm{x}, \mathrm{w})}{\partial w_i}$$

We need to compute the **partial derivative of the network** $y(\mathrm{x}, \mathrm{w})$ with respect to **each weight** w_i

Numerical Differentiation

Proposal: **Numerical Differentiation**

3940975

We could estimate the gradient numerically by computing:

$$\frac{\partial}{\partial w_i} E(\mathrm{w}; \mathcal{D}) \approx \frac{E(\mathrm{w} + \epsilon\, I_i; \mathcal{D}) + E(\mathrm{w} - \epsilon\, I_i; \mathcal{D})}{2\epsilon}$$

for a small value of ϵ (where I_i is the i^{th} column of the identity matrix).

```python
def numeric_gradient(f, x1, x2, h=1e-8):
    """
    Compute the numerical derivative of f with respect to x1 at (x1, x2)
    using central difference.
    """
    return [(f(x1 + h, x2) - f(x1 - h, x2)) / (2 * h),
            (f(x1, x2 + h) - f(x1, x2 - h)) / (2 * h)]
```

✓ 0.0s

```python
numeric_gradient(f, 1, 2)
```

✓ 0.0s

```
[np.float64(16.778112232884723), np.float64(8.80520287793729)]
```

Proposal: **Numerical Differentiation**

Advantage?

- Can apply to ***any*** **error function** and network architecture.
- Easy to implement.

Disadvantage?

- Approximate – but likely to be **fairly accurate** for small ϵ.
- Expensive! – Why?

Cost of Numerical Differentiation

Computing the **gradient** using numerical differentiation:

$$\frac{\partial}{\partial w_i} E(\mathrm{w}; \mathcal{D}) \approx \frac{E(\mathrm{w} + \epsilon\, I_i; \mathcal{D}) + E(\mathrm{w} - \epsilon\, I_i; \mathcal{D})}{2\epsilon}$$

for a D dimensional weight vector $\mathrm{w} \in \mathbb{R}^D$ requires:

$$\left[\frac{\partial}{\partial w_1} E(\mathrm{w}; \mathcal{D}), \ldots, \frac{\partial}{\partial w_D} E(\mathrm{w}; \mathcal{D})\right]$$

$2D$ error calculations. Recall each **error calculation** requires $\mathcal{O}(ND)$ computation for N datapoints.

- **Total cost:** $\mathcal{O}(ND^2)$ per gradient step!
  - For real problems (minibatch) often $N \approx 1000$ and $D \gg 10^6$ → $ND^2 \gg 10^{3+2*6} = 10^{15}$
  - Stochastic gradient with batch size 1: $D^2 \gg 10^{12}$

Symbolic Differentiation

Proposal: **Symbolic Differentiation**

Continue to use calculus to derive the gradient equation.

$$\frac{\partial}{\partial w_i} E(\mathrm{w}; \mathcal{D}) = -\sum_{d=1}^{D} t_d \left(\frac{1}{y(\mathrm{x}, \mathrm{w})}\right) \frac{\partial y(\mathrm{x}, \mathrm{w})}{\partial w_i} + (1 - t_d) \left(\frac{1}{1 - y(\mathrm{x}, \mathrm{w})}\right) \frac{\partial y(\mathrm{x}, \mathrm{w})}{\partial w_i}$$

for our small network: $y(\mathrm{x}, \mathrm{w}) = \sigma\left(\mathrm{W}^{(2)} \sigma\left(\mathrm{W}^{(1)} x\right)\right)$ and using the identity:

$$\frac{\partial}{\partial t} \sigma(t) = \sigma(t)(1 - \sigma(t))$$

Then taking the partial derivative of the ij^{th} entry of $\mathrm{W}^{(1)}$ we get:

$$\frac{\partial}{\partial W_{ij}^{(1)}} y(\mathrm{x}, \mathrm{w}) = \sigma\left(\mathrm{W}^{(2)} \sigma\left(\mathrm{W}^{(1)} x\right)\right)\left(1 - \sigma\left(\mathrm{W}^{(2)} \sigma\left(\mathrm{W}^{(1)} x\right)\right)\right) \mathrm{W}^{(2)} \frac{\partial}{\partial W_{ij}^{(1)}} \sigma\left(\mathrm{W}^{(1)} x\right)$$

$$\frac{\partial}{\partial W_{i}^{(1)}} y(\mathrm{x}, \mathrm{w}) = y(\mathrm{x}, \mathrm{w})(1 - y(\mathrm{x}, \mathrm{w})) \mathrm{W}_{\mathrm{j}}^{(2)} \sigma\left(\mathrm{W}^{(1)} x\right)\left(1 - \sigma\left(\mathrm{W}^{(1)} x\right)\right) x_i$$

Painful and error prone (there is probably a mistake on this slide?) – **Not automatic** .

Symbolic Computer Algebra Systems

Computer algebra systems like SymPy can be used to automate the differentiation process.

- Express functions **symbolically**
- Automatically construct **derivative expressions**

```
n = sp.symbols('n', integer=True)    # number of data points (symbolic length)
i = sp.Idx('i', n)                   # index variable for summation
w0, w1 = sp.symbols('w0 w1')         # weights
x = sp.IndexedBase('x', shape=(n,))  # indexed variable for x data
y = sp.IndexedBase('y', shape=(n,))  # indexed variable for y data

E_i = (y[i] - sp.sin(w0 + w1 * x[i]))**2
E = sp.summation(E_i, (i, 0, n - 1)) / n

gE = [sp.diff(E, var) for var in (w0, w1)]
gE
```

✓ 0.0s Python

```
[Sum(-2*(-sin(w0 + w1*x[i]) + y[i])*cos(w0 + w1*x[i]), (i, 0, n - 1))/n,
 Sum(-2*(-sin(w0 + w1*x[i]) + y[i])*cos(w0 + w1*x[i])*x[i], (i, 0, n - 1))/n]
```

Demo

SymPy

3940975

```
import sympy as sp
# define our symbols
x1, x2 = sp.symbols('x1 x2')
# Define a symbolic expression for the error
E = x1 * x2 + sp.exp(x1 * x2) - sp.sin(x2)
# Compute the gradient of E with respect to x1 and x2
gE = [sp.diff(E, var) for var in (x1, x2)]
gE
```

Symbolic Computer Algebra Systems

3940975

Computer algebra systems like SymPy can be used to automate the differentiation process.

- Express functions **symbolically**
- Automatically construct **derivative expressions**

Advantages:

- **Automatic:** algebraic systems can be applied to "any" expression
  - Some issues with error functions that contain control flow/loops.
- **Exact:** Doesn't require epsilon approximation

Disadvantages:

- **Inefficient: a**lgebraic derivative expressions could be significantly larger and contain many repeated expressions:
  - **Why?**

Expression Swell

Consider computing the derivative of an expression like:

$$f(x) = g(x)h(x)u(x)$$

If we apply the product rule, we get:

$$\frac{\partial}{\partial x} f(x) = h(x)u(x)\frac{\partial}{\partial x} g(x) + g(x)u(x)\frac{\partial}{\partial x} h(x) + g(x)h(x)\frac{\partial}{\partial x} u(x)$$

Notice that there is **significant redundancy**. This is especially the case with neural networks (from earlier):

$$\frac{\partial}{\partial W_i^{(1)}} y(\mathrm{x}, \mathrm{w}) = y(\mathrm{x}, \mathrm{w})\left(1 - y(\mathrm{x}, \mathrm{w})\right) \mathrm{W}_\mathrm{j}^{(2)} \sigma\left(\mathrm{W}^{(1)} x\right)\left(1 - \sigma\left(\mathrm{W}^{(1)} x\right)\right) x_i$$

$$\sigma\left(\mathrm{W}^{(2)} \sigma\left(\mathrm{W}^{(1)} x\right)\right)$$

Automatic Differentiation

Automatic Differentiation (Autodiff)

3940975

Automatically generate **a program** to compute the derivative **efficiently** from the **"forward" computation** of the the error.

- **Automatic:** derivative program is constructed by tracing the forward computation graph.
- **Accurate:** derivative is computed to machine precision.
- **Efficient:** reuse redundant computation

There are two approaches to Autodiff:

- **Forward Mode:** Best for $f: \mathbb{R} \to \mathbb{R}^D$
- **Backward Mode:** Best for $f: \mathbb{R}^D \to \mathbb{R}^1$ (deep learning)

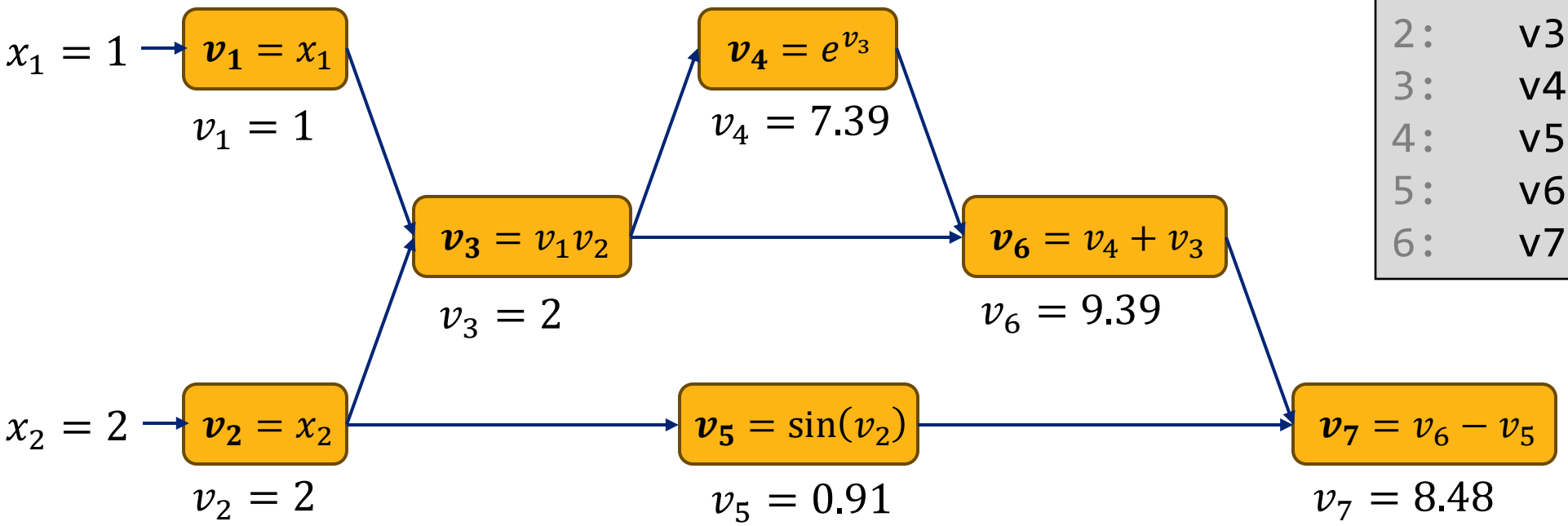
Forward Tracing the Computation Graph

We construct the graph by evaluating the forward computation:

$$f(x_1, x_2) = x_1 x_2 + e^{(x_1 x_2)} - \sin(x_2)$$

Program:

```
0:    v1 = x1
1:    v2 = x2
2:    v3 = v1 * v2
3:    v4 = exp(v3)
4:    v5 = sin(v2)
5:    v6 = v4 + v3
6:    v7 = v6 - v5
```

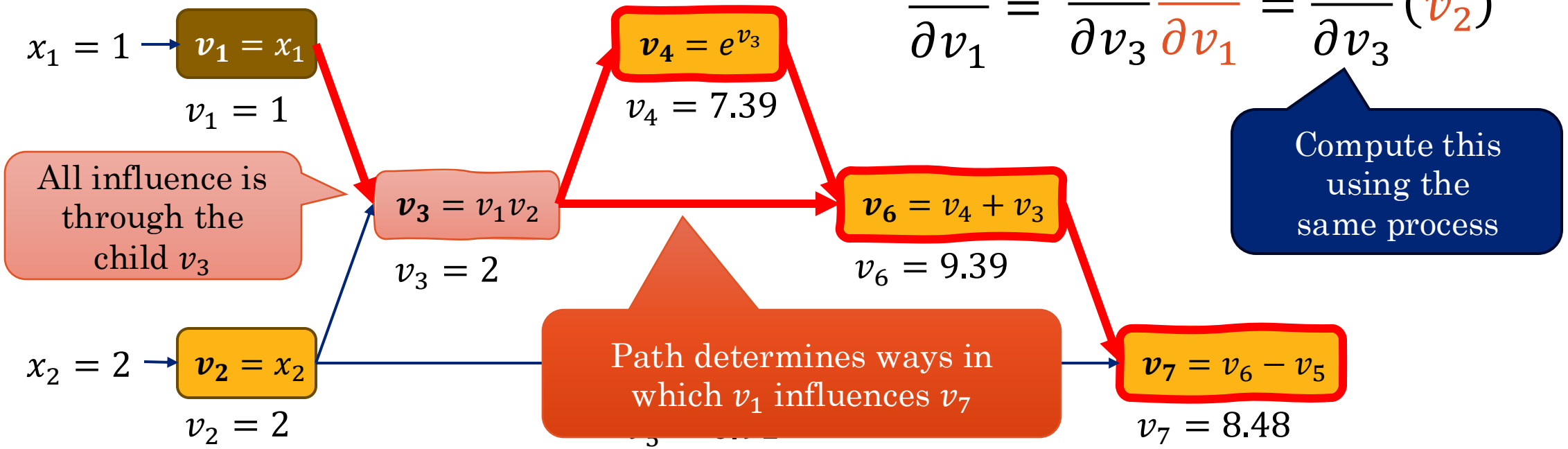


In this program we have broken each expression into a separate line.

Differentiating wrt v_1 Using the Chain Rule

Computing the derivative of $\frac{\partial f}{\partial v_1}$ by observing the **path** from v_1 to v_7.

$$\frac{\partial f}{\partial v_1} = \frac{\partial f}{\partial v_3}\frac{\partial v_3}{\partial v_1} = \frac{\partial f}{\partial v_3}(v_2)$$



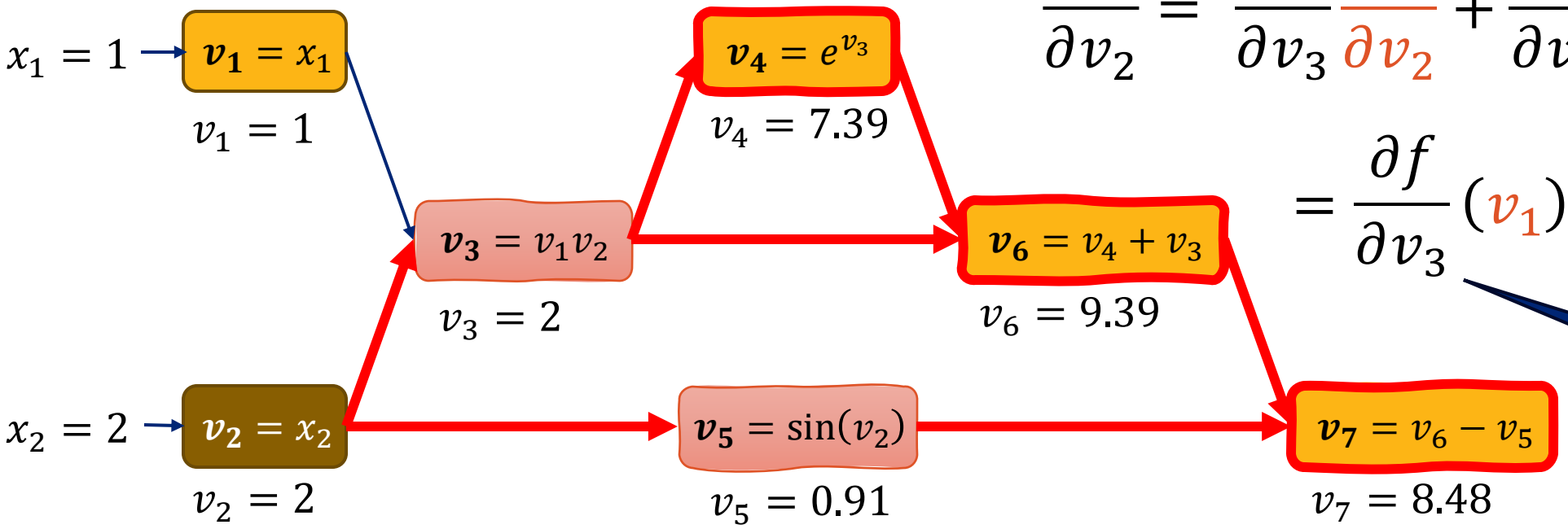
$$f(x_1, x_2) = x_1 x_2 + e^{(x_1 x_2)} - \sin(x_2)$$

Differentiating wrt v_2 Using the Chain Rule

Computing the derivative of $\frac{\partial f}{\partial v_2}$ by observing the paths from v_2 to v_7.

$$\frac{\partial f}{\partial v_2} = \frac{\partial f}{\partial v_3}\frac{\partial v_3}{\partial v_2} + \frac{\partial f}{\partial v_5}\frac{\partial v_5}{\partial v_2}$$

$$= \frac{\partial f}{\partial v_3}(v_1) + \frac{\partial f}{\partial v_5}\cos(v_2)$$



Compute these using the same process

$$f(x_1, x_2) = x_1 x_2 + e^{(x_1 x_2)} - \sin(x_2)$$

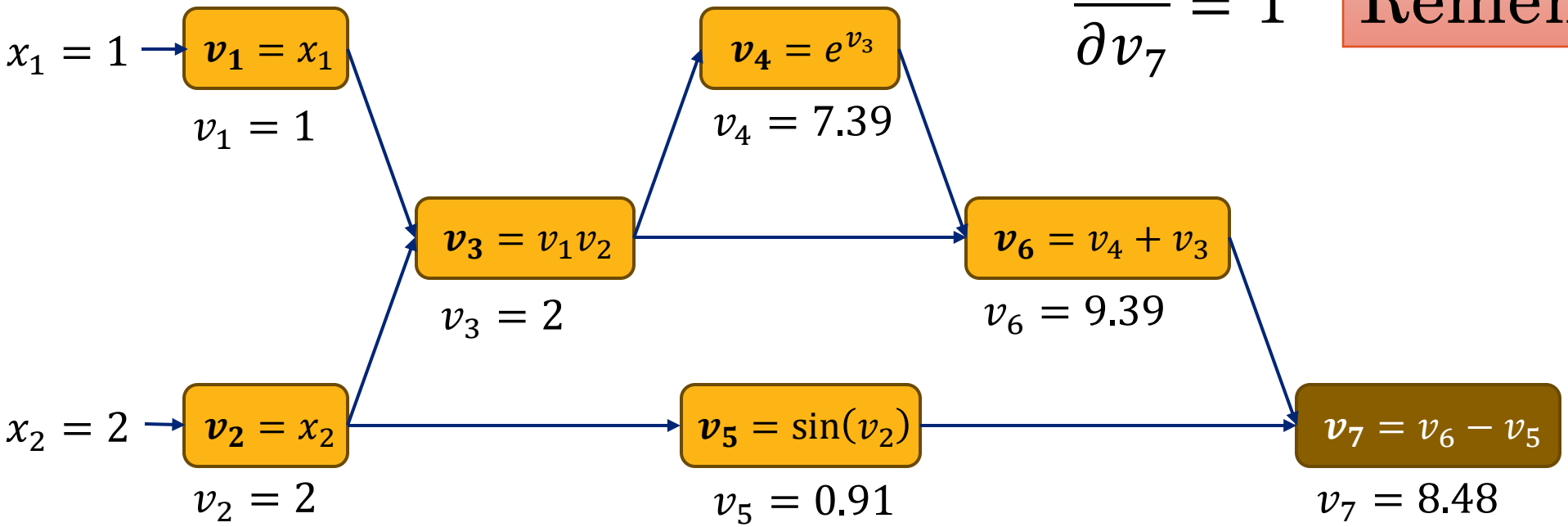
Differentiating wrt v_7 Using the Chain Rule

Computing the derivative of $\frac{\partial f}{\partial v_7}$ by observing the paths from v_7 to v_7.

$$\frac{\partial f}{\partial v_7} = 1$$

Remember $f = v_7$

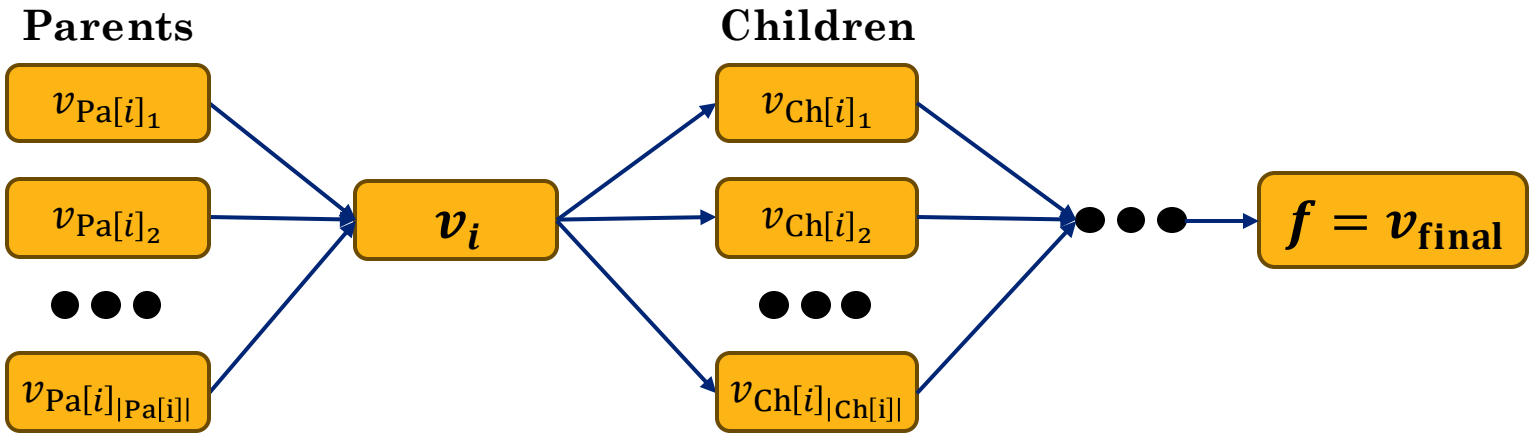
Base Case!



$$f(x_1, x_2) = x_1 x_2 + e^{(x_1 x_2)} - \sin(x_2)$$

Backward Propagation (Recursion)

Consider the computation graph around $\boldsymbol{v_i}$ with a **single output** $\boldsymbol{f = v_{\text{final}}}$



We introduce the **adjoint variable** $\boldsymbol{a_i^f}$ and use the **chain rule to obtain**:

Recursion:

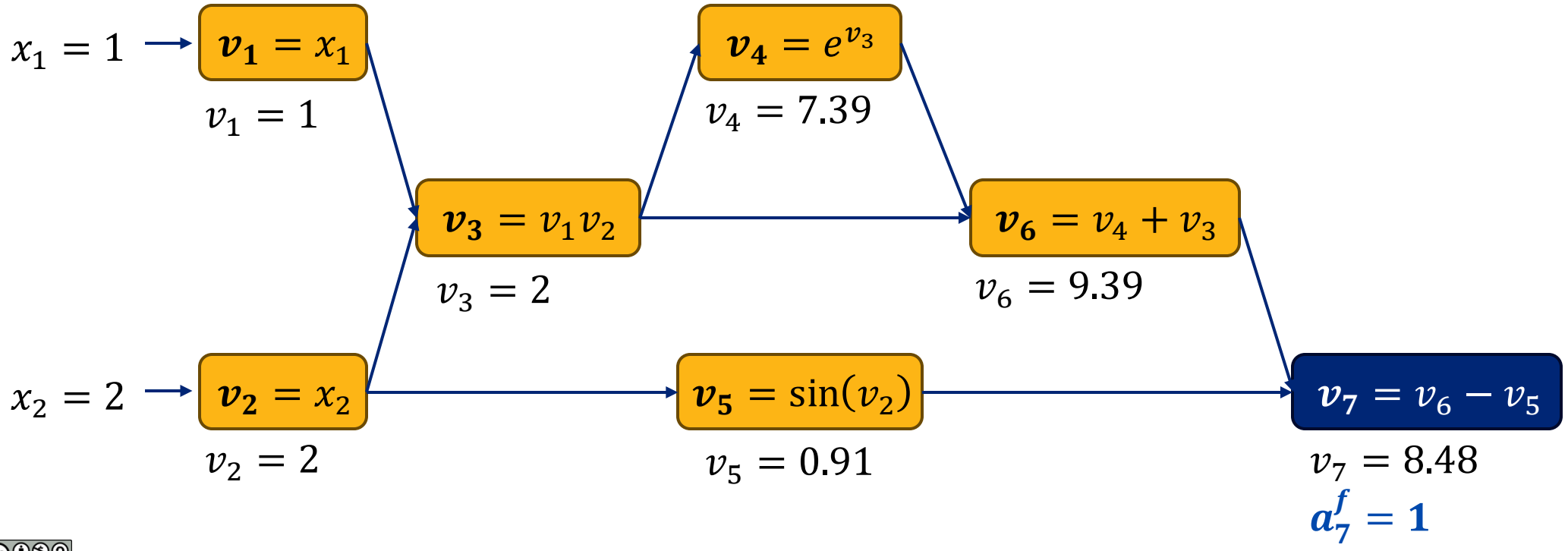
$$a_i^f = \frac{\partial f}{\partial v_i} = \sum_{j \in \mathbf{Ch}[i]} \left(\frac{\partial f}{\partial v_j}\right)\left(\frac{\partial v_j}{\partial v_i}\right) = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i}\right)$$

Where $\mathbf{Ch}[i]$ is the children of vertex i in the computation graph.

Backward Autodiff Example

$$a_i^f = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward.**

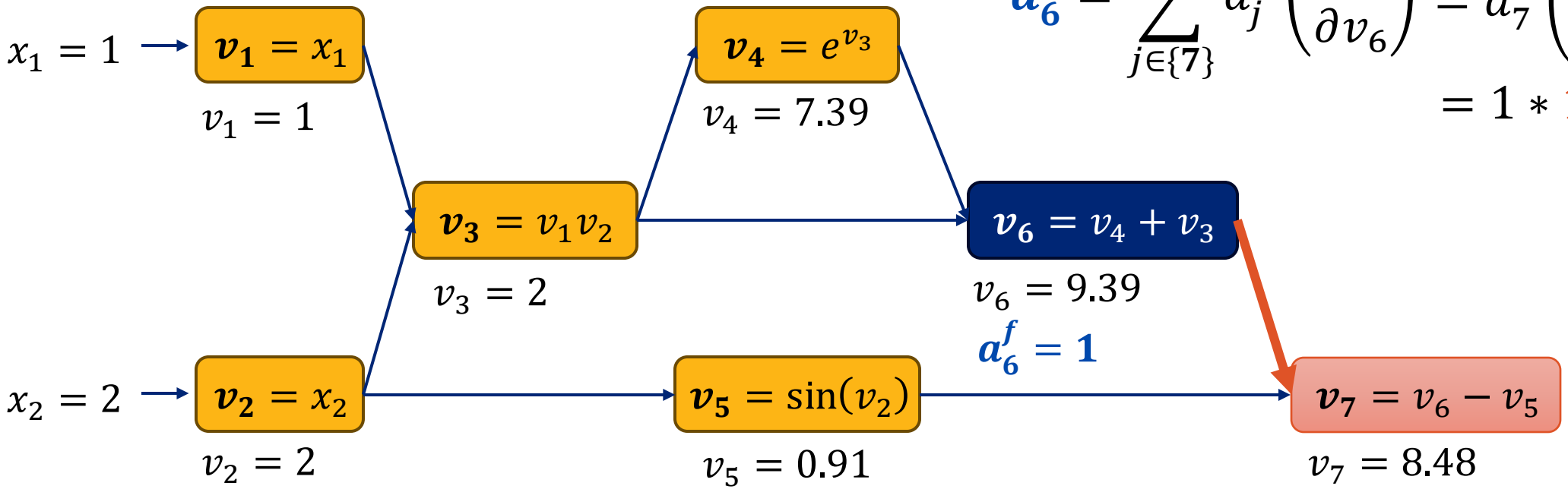


Backward Autodiff a_6^f

$$a_i^f = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward.**

$$a_6^f = \sum_{j \in \{7\}} a_j^f \left(\frac{\partial v_j}{\partial v_6} \right) = a_7^f \left(\frac{\partial v_7}{\partial v_6} \right) = 1 * 1$$



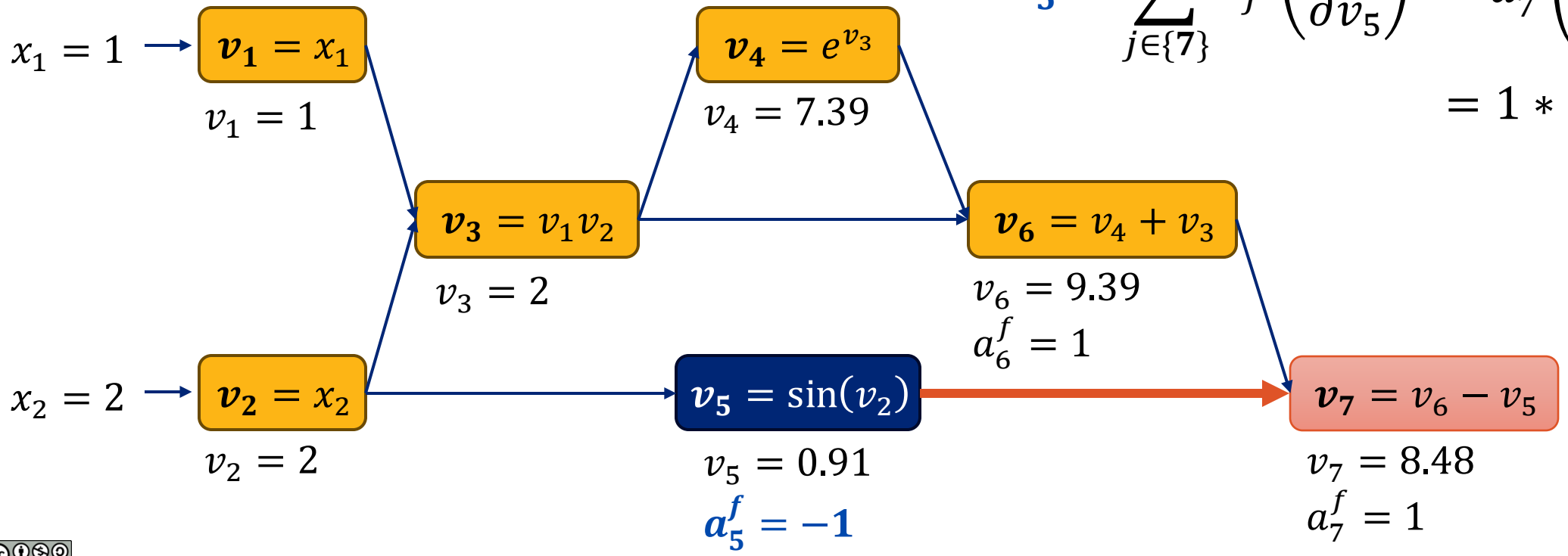
Backward Autodiff a_5^f

$$a_i^f = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward.**

$$a_5^f = \sum_{j \in \{7\}} a_j^f \left(\frac{\partial v_j}{\partial v_5} \right) = a_7^f \left(\frac{\partial v_7}{\partial v_5} \right)$$

$$= 1 * (-1)$$



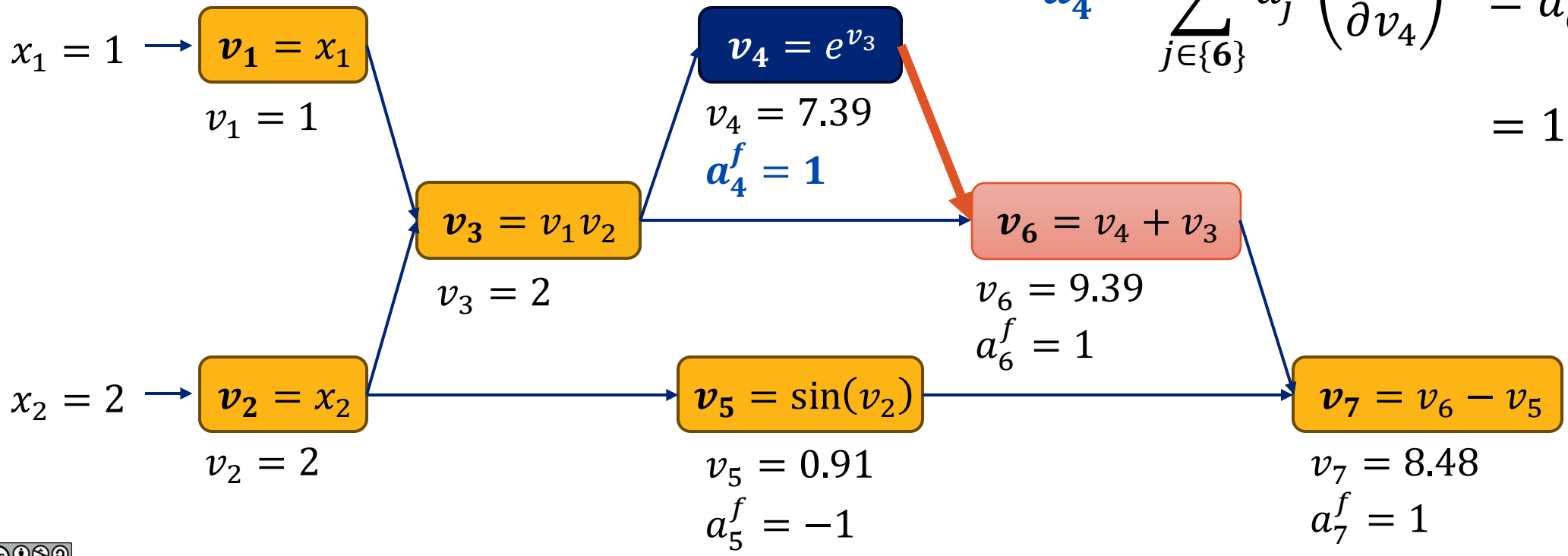
Backward Autodiff a_4^f

$$a_i^f = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward.**

$$a_4^f = \sum_{j \in \{6\}} a_j^f \left(\frac{\partial v_j}{\partial v_4} \right) = a_6^f \left(\frac{\partial v_6}{\partial v_4} \right)$$

$$= 1 * (1)$$

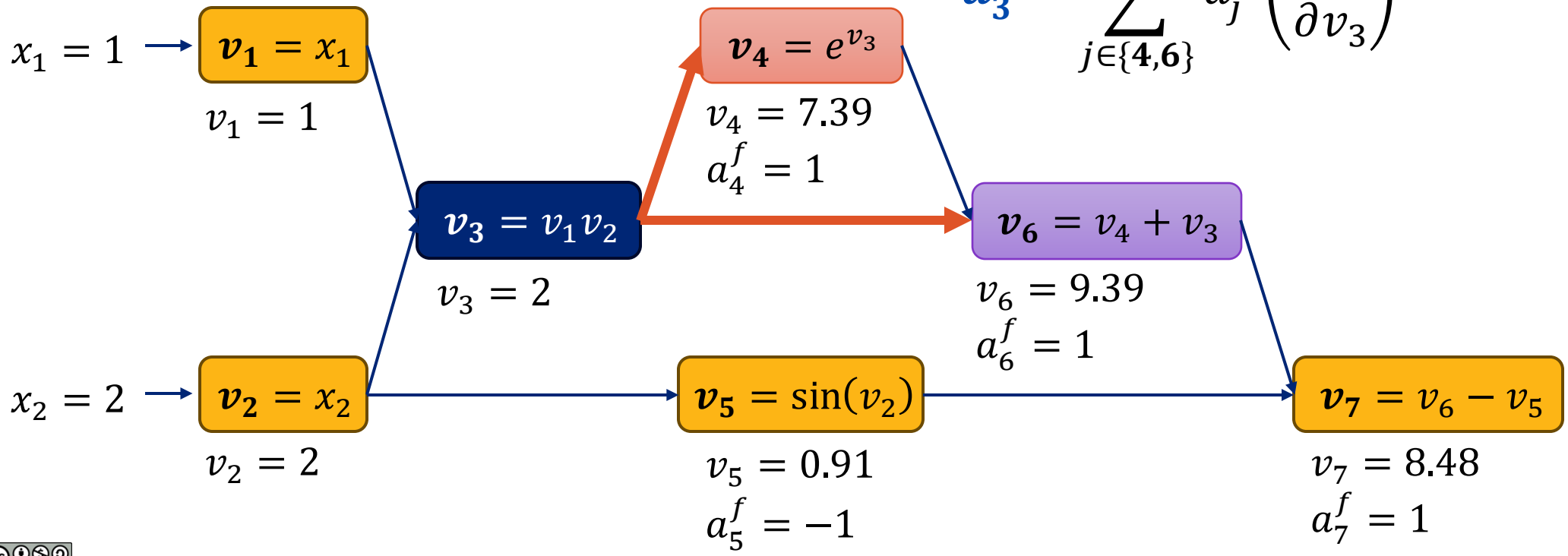


Backward Autodiff a_3^f

$$a_i^f = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward.**

$$a_3^f = \sum_{j \in \{4,6\}} a_j^f \left(\frac{\partial v_j}{\partial v_3} \right)$$

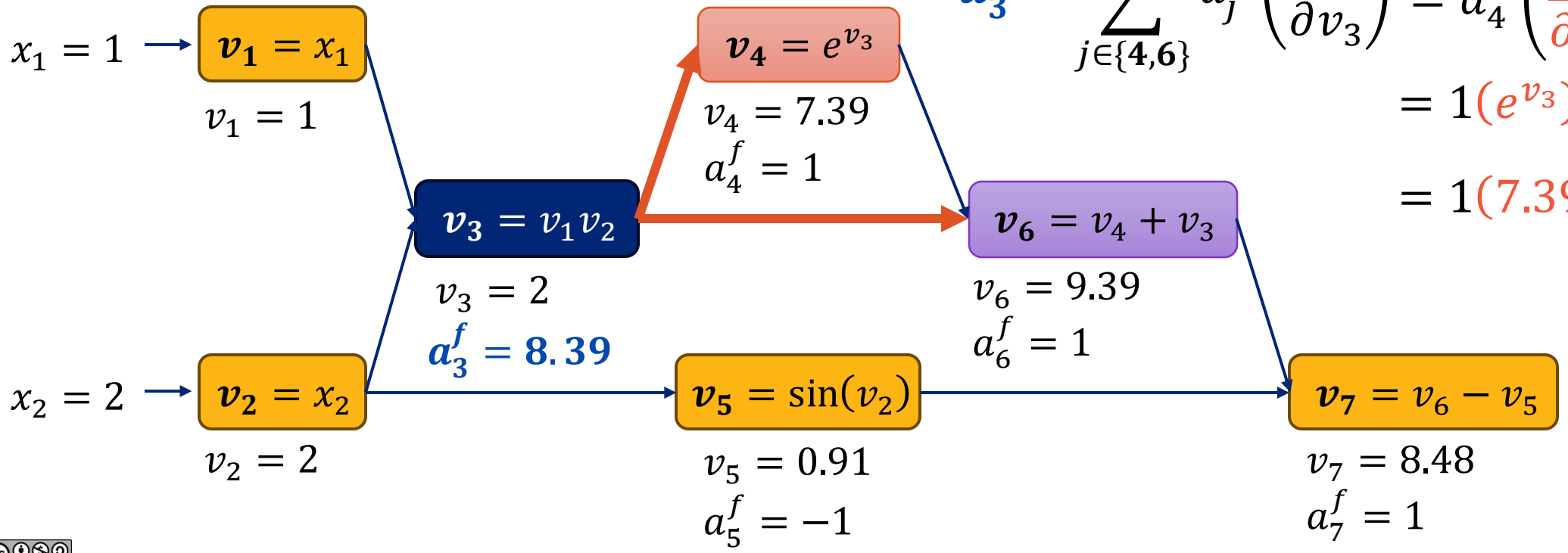


What is the value of the adjoint variable a_3.

Backward Autodiff a_3^f

$$a_i^f = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward.**



$$\boldsymbol{a_3^f} = \sum_{j \in \{\mathbf{4},\mathbf{6}\}} a_j^f \left(\frac{\partial v_j}{\partial v_3} \right) = a_4^f \left(\frac{\partial v_4}{\partial v_3} \right) + a_6^f \left(\frac{\partial v_6}{\partial v_3} \right)$$

$$= 1(e^{v_3}) + 1(1)$$

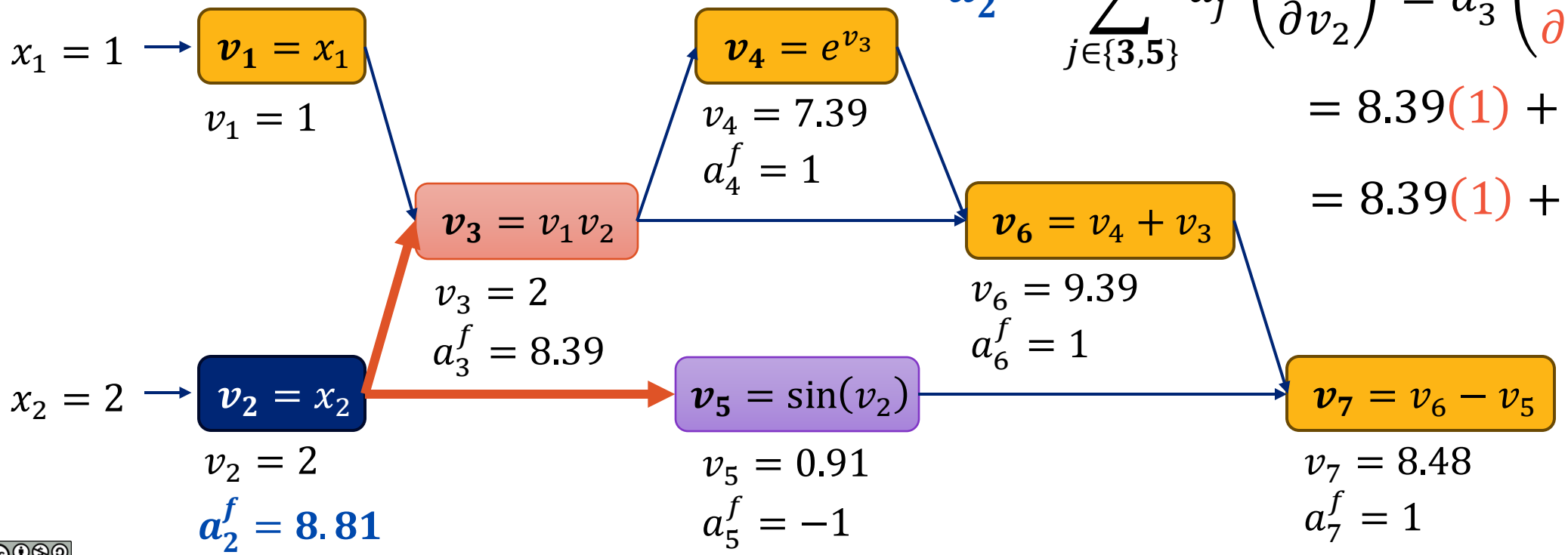
$$= 1(7.39) + 1(1)$$

3940975

Backward Autodiff a_2^f

$$a_i^f = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward.**



$$\boldsymbol{a_2^f} = \sum_{j \in \{\mathbf{3},\mathbf{5}\}} a_j^f \left(\frac{\partial v_j}{\partial v_2} \right) = a_3^f \left(\frac{\partial v_3}{\partial v_2} \right) + a_5^f \left(\frac{\partial v_5}{\partial v_2} \right)$$

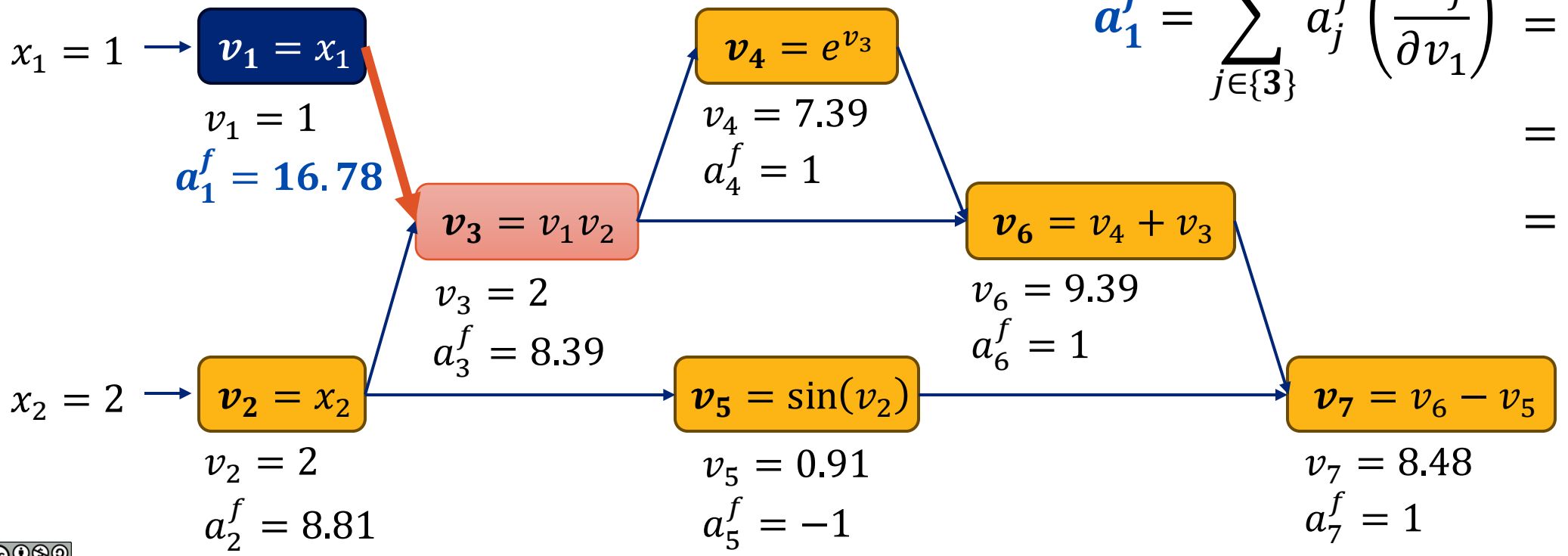
$$= 8.39(1) + (-1)(\cos(v_2))$$

$$= 8.39(1) + (-1)(-0.42)$$

Backward Autodiff a_1^f

$$a_i^f = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward.**



$$a_1^f = \sum_{j \in \{3\}} a_j^f \left(\frac{\partial v_j}{\partial v_1} \right) = a_3^f \left(\frac{\partial v_3}{\partial v_1} \right)$$

$$= 8.39\ (v_2)$$

$$= 8.39\ (2)$$

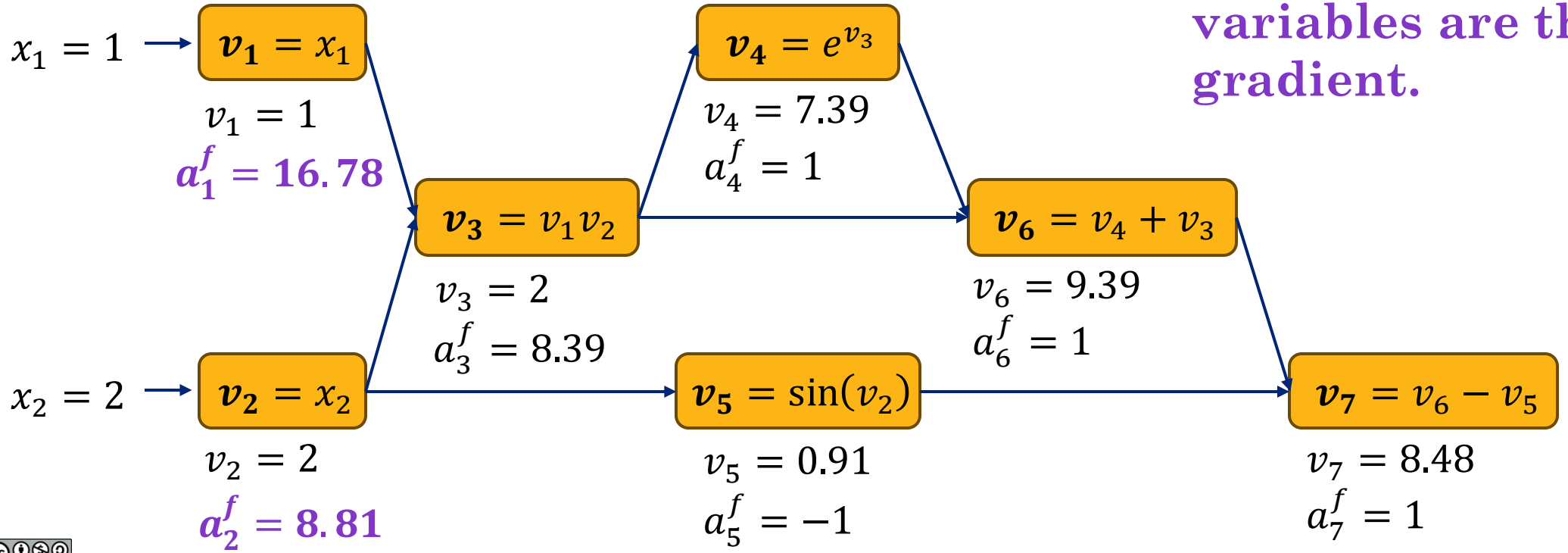
3940975

Obtaining The Gradient!

$$a_i^f = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

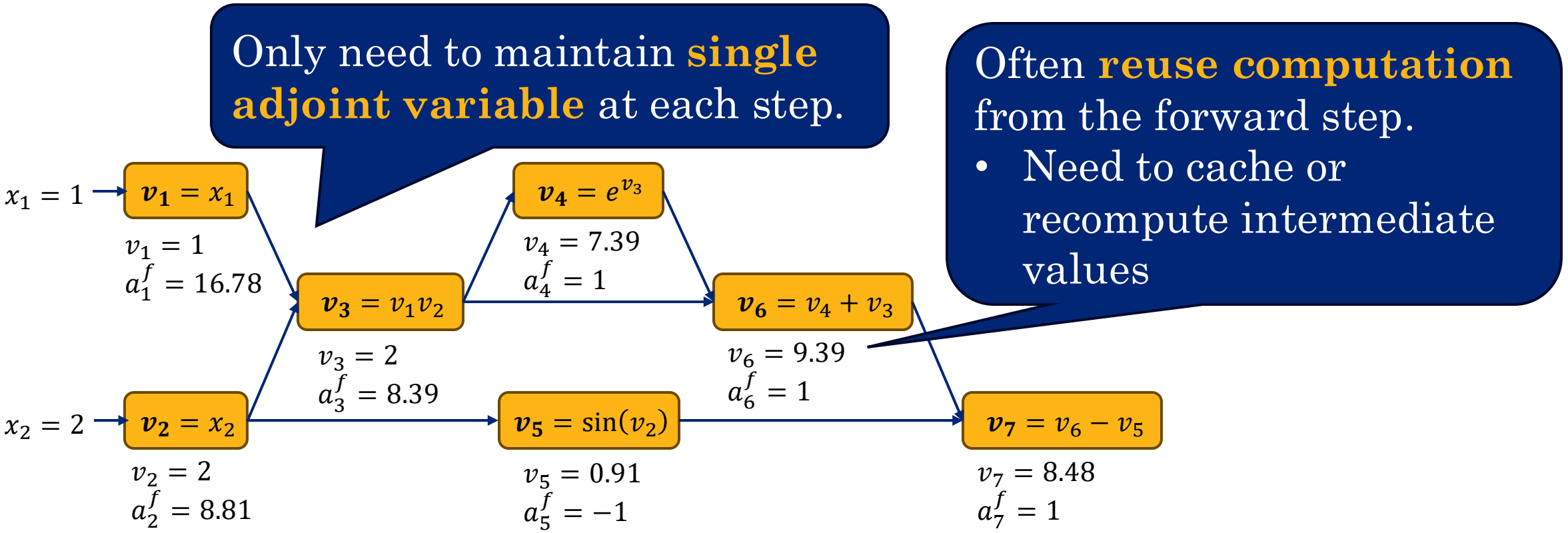
1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward.**

➡ **3. The final input adjoint variables are the gradient.**



Cost Analysis

Backpropagation is efficient when there is a single output and many inputs.

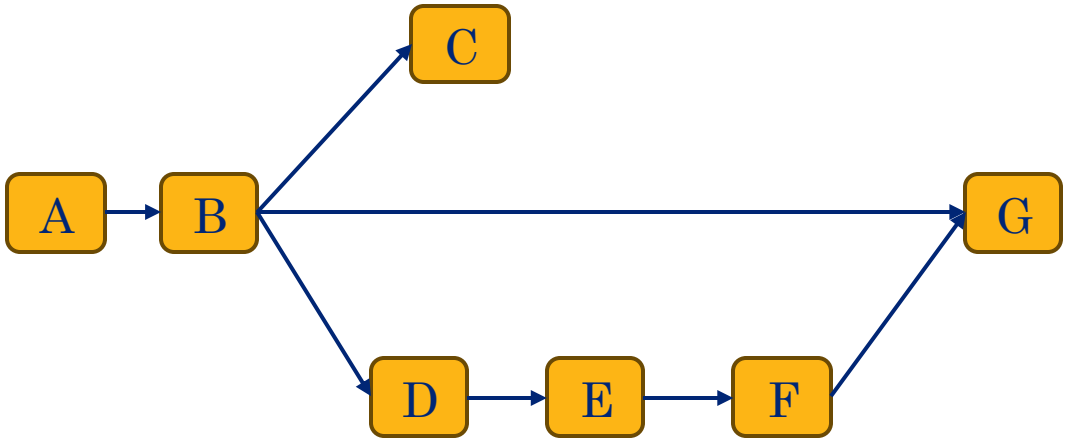


Need to run **backpropagation repeatedly** for **multiple outputs**.

Demo

You will do this in your homework.

- But my example code (and my example bug) may be helpful.



Hint:
The **adjoint at B** can't be computed until the **adjoint at D is computed** (*topological ordering*) but does not depend on the **adjoint at C** (*an inactive path*).

3940975

Additional Reading

3940975

- Automatic Differentiation in Machine Learning: A Survey (2018)
- A Gentle Introduction to torch.autograd

Lecture 15

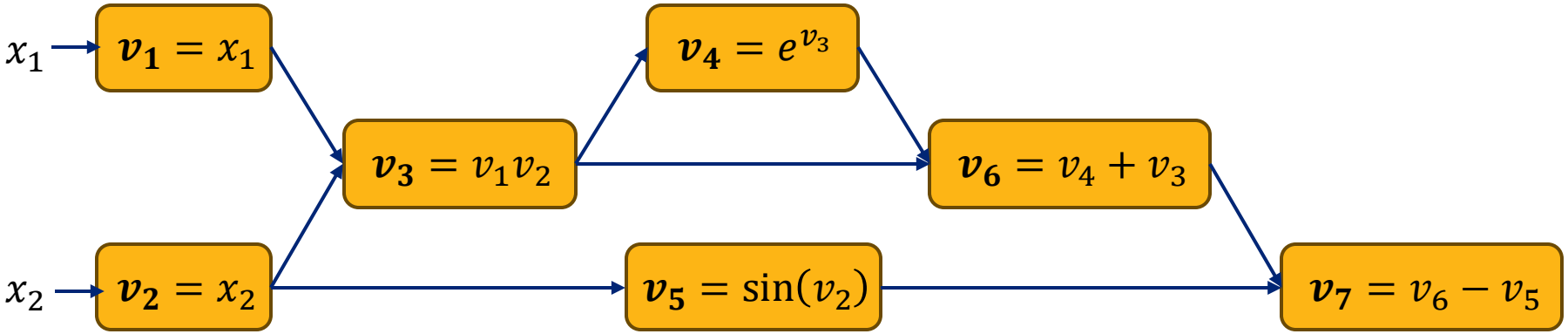
PyTorch Continued & Backpropagation

Reading: *Chapter 8* in Bishop Deep Learning Textbook

Bonus Material

Forward Differentiation

Chain Rule on The Computation Graph



If we want to compute the derivative $\frac{\partial}{\partial x_2} f$ using the chain rule:

$$\frac{\partial v_7}{\partial x_2} = \frac{\partial v_7}{\partial v_6}\left(\frac{\partial v_6}{\partial x_2}\right) + \frac{\partial v_7}{\partial v_5}\left(\frac{\partial v_5}{\partial x_2}\right) = \left(\frac{\partial v_6}{\partial x_2}\right) - \left(\frac{\partial v_5}{\partial x_2}\right)$$

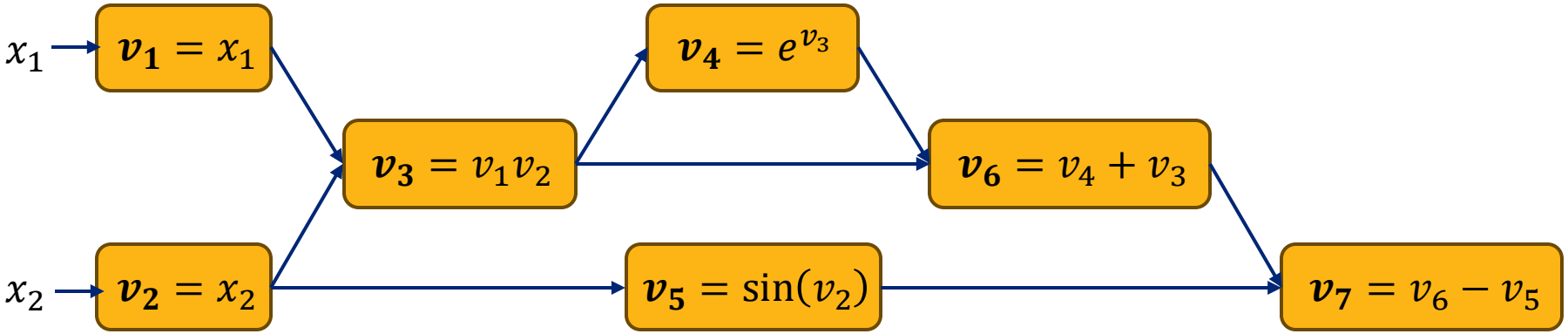
$$\frac{\partial v_3}{\partial x_2} = \frac{\partial v_3}{\partial v_1}\left(\frac{\partial v_1}{\partial x_2}\right) + \frac{\partial v_3}{\partial v_2}\left(\frac{\partial v_2}{\partial x_2}\right) = v_2\left(\frac{\partial v_1}{\partial x_2}\right) + v_1\left(\frac{\partial v_2}{\partial x_2}\right)$$

We can generalize these expressions with:

$$\frac{\partial v_i}{\partial x_2} = \sum_{j \in \mathbf{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j}\right)\left(\frac{\partial v_j}{\partial x_2}\right)$$

where $\mathbf{Pa}[i]$ are the parents of i in the computation graph.

Chain Rule on The Computation Graph



If we define a new **tangent variable** representing the partial derivative $\partial_{x_k}^i = \frac{\partial v_i}{\partial x_k}$

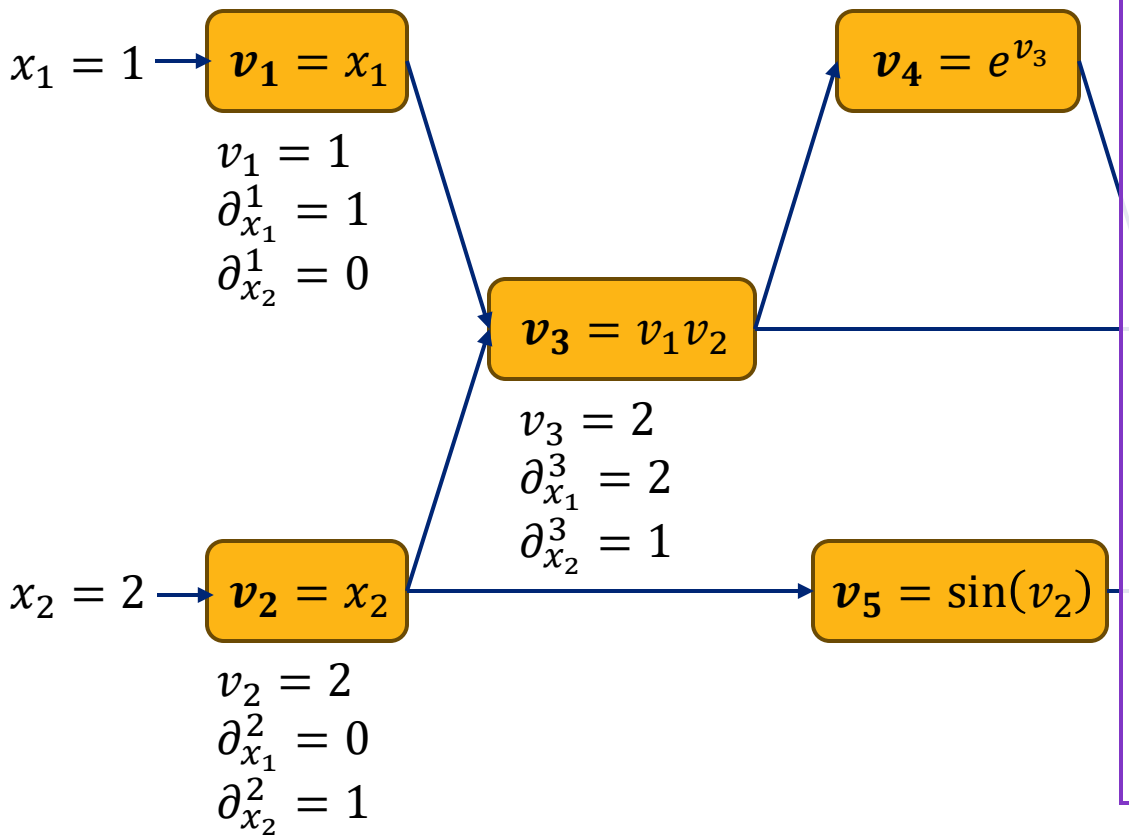
$$\partial_{x_k}^i = \frac{\partial v_i}{\partial x_k} = \sum_{j \in \mathbf{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j}\right)\left(\frac{\partial v_j}{\partial x_k}\right) = \sum_{j \in \mathbf{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j}\right) \partial_{x_k}^j$$

During the **forward computation** we can compute the value of v_i and $\partial_{x_k}^i$:

- Notice that v_i and $\partial_{x_2}^i$ only depend on their parents.
- Need to maintain separate $\partial_{x_k}^i$ for each variable k

Forward Autodiff Example

$$\partial^i_{x_k} = \sum_{j \in \mathbf{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \partial^j_{x_k}$$



Derivation

$$\partial^3_{x_1} = \sum_{j \in \{\mathbf{1},\mathbf{2}\}} \left(\frac{\partial v_3}{\partial v_j} \right) \partial^j_{x_1} = v_2 \partial^1_{x_1} + v_1 \partial^2_{x_1}$$
$$= 2 * 1 + 1 * 0$$
$$= 2$$

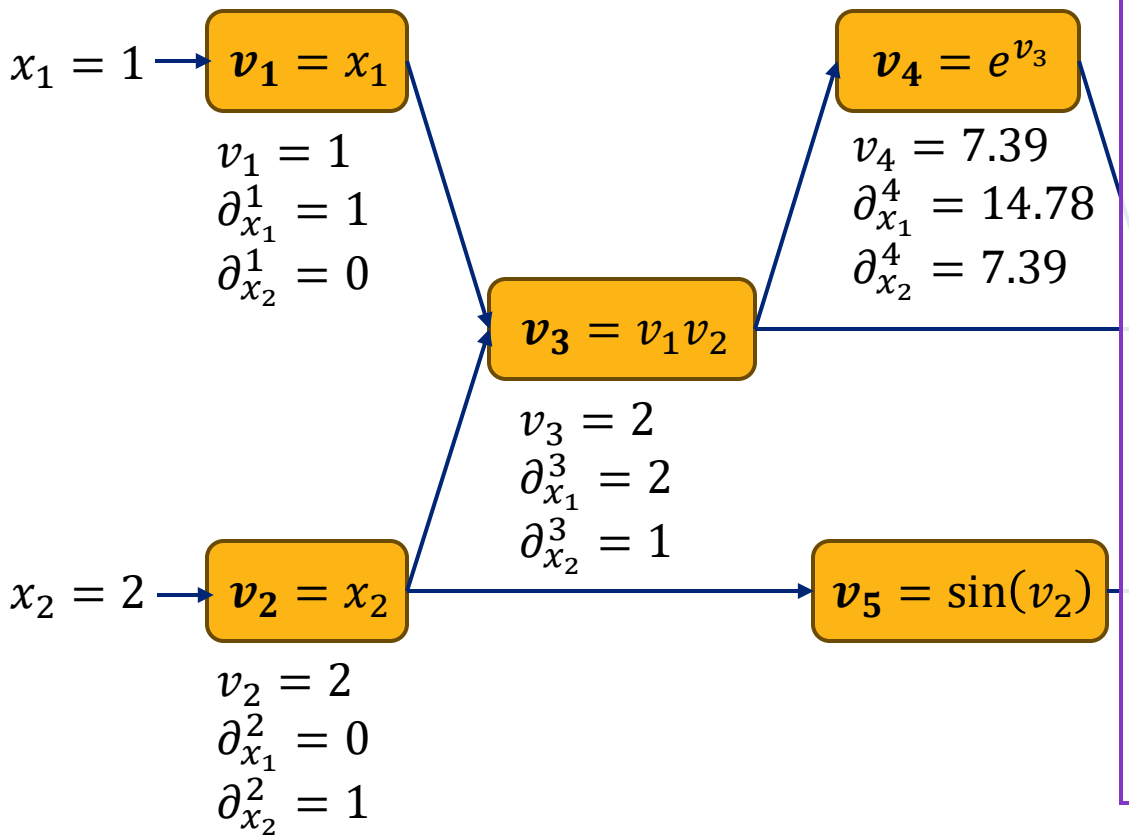
$v_6 = v_4 + v_3$

$$\partial^3_{x_2} = \sum_{j \in \{\mathbf{1},\mathbf{2}\}} \left(\frac{\partial v_3}{\partial v_j} \right) \partial^j_{x_2} = v_2 \partial^1_{x_2} + v_1 \partial^2_{x_2}$$
$$= 2 * 0 + 1 * 1$$
$$= 1$$

$v_7 = v_6 - v_5$

Forward Autodiff Example

$$\partial^i_{x_k} = \sum_{j \in \mathbf{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \partial^j_{x_k}$$



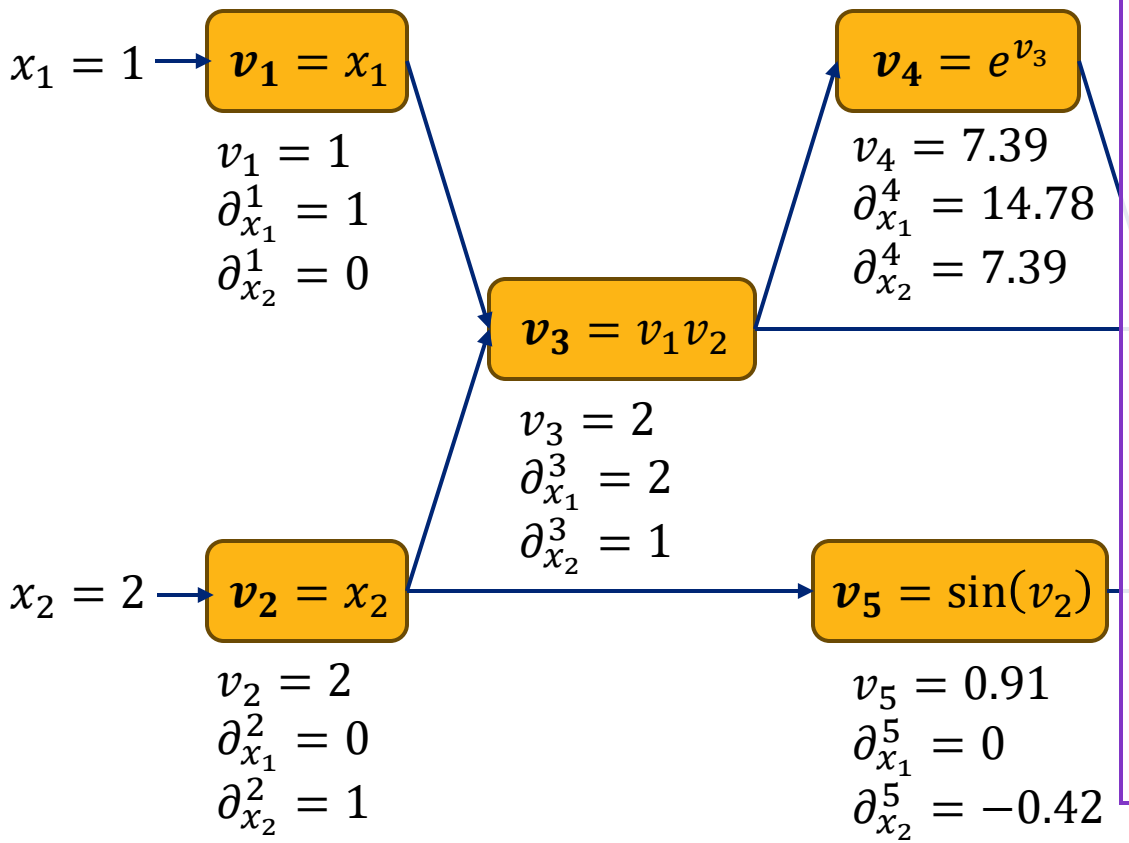
Derivation

$$\partial^4_{x_1} = \sum_{j \in \{\mathbf{3}\}} \left(\frac{\partial v_4}{\partial v_j} \right) \partial^j_{x_1} = \left(\frac{\partial v_4}{\partial v_3} e^{v_3} \right) \partial^3_{x_1}$$
$$= (e^{v_3}) \partial^3_{x_1}$$
$$= (v_4)\, 2 = 14.78$$

$$\partial^4_{x_2} = \sum_{j \in \{\mathbf{3}\}} \left(\frac{\partial v_4}{\partial v_j} \right) \partial^j_{x_2} = \left(\frac{\partial v_4}{\partial v_3} e^{v_3} \right) \partial^3_{x_2}$$
$$= (v_4) \partial^3_{x_2}$$
$$= (v_4)\, 1 = 7.39$$

Forward Autodiff Example

$$\partial^i_{x_k} = \sum_{j \in \mathbf{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j}\right) \partial^j_{x_k}$$



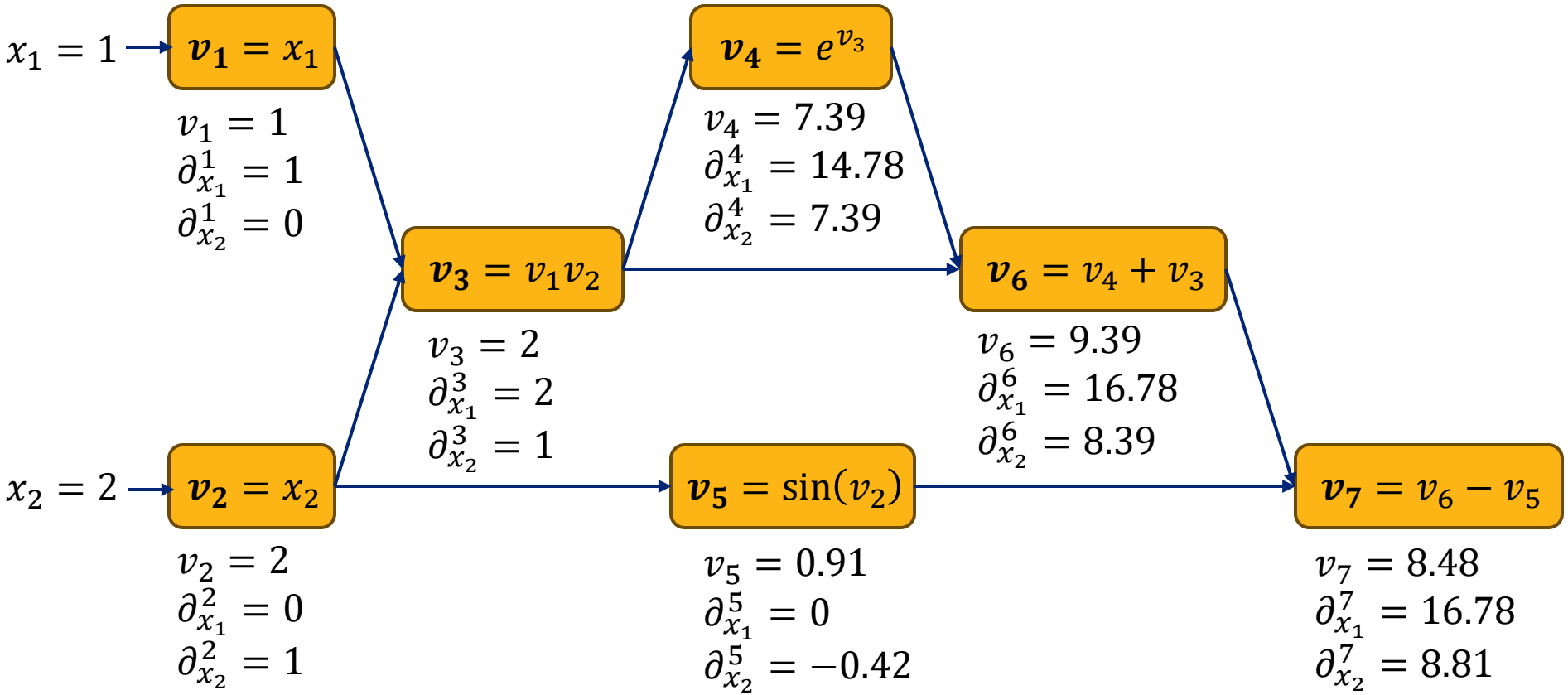
Derivation

$$\partial^5_{x_1} = \sum_{j \in \{2\}} \left(\frac{\partial v_5}{\partial v_j}\right) \partial^j_{x_1} = \left(\frac{\partial v_5}{\partial v_2} \sin(v_2)\right) \partial^2_{x_1}$$
$$= (\cos(v_2))\partial^2_{x_1}$$
$$= (-0.42)0 = 0$$

$$\partial^5_{x_2} = \sum_{j \in \{2\}} \left(\frac{\partial v_5}{\partial v_j}\right) \partial^j_{x_2} = \left(\frac{\partial v_5}{\partial v_2} \sin(v_2)\right) \partial^2_{x_2}$$
$$= (\cos(v_2))\partial^2_{x_2}$$
$$= (-0.42)1 = -0.42$$

Forward Autodiff Example

$$\partial^i_{x_k} = \sum_{j \in \mathbf{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \partial^j_{x_k}$$

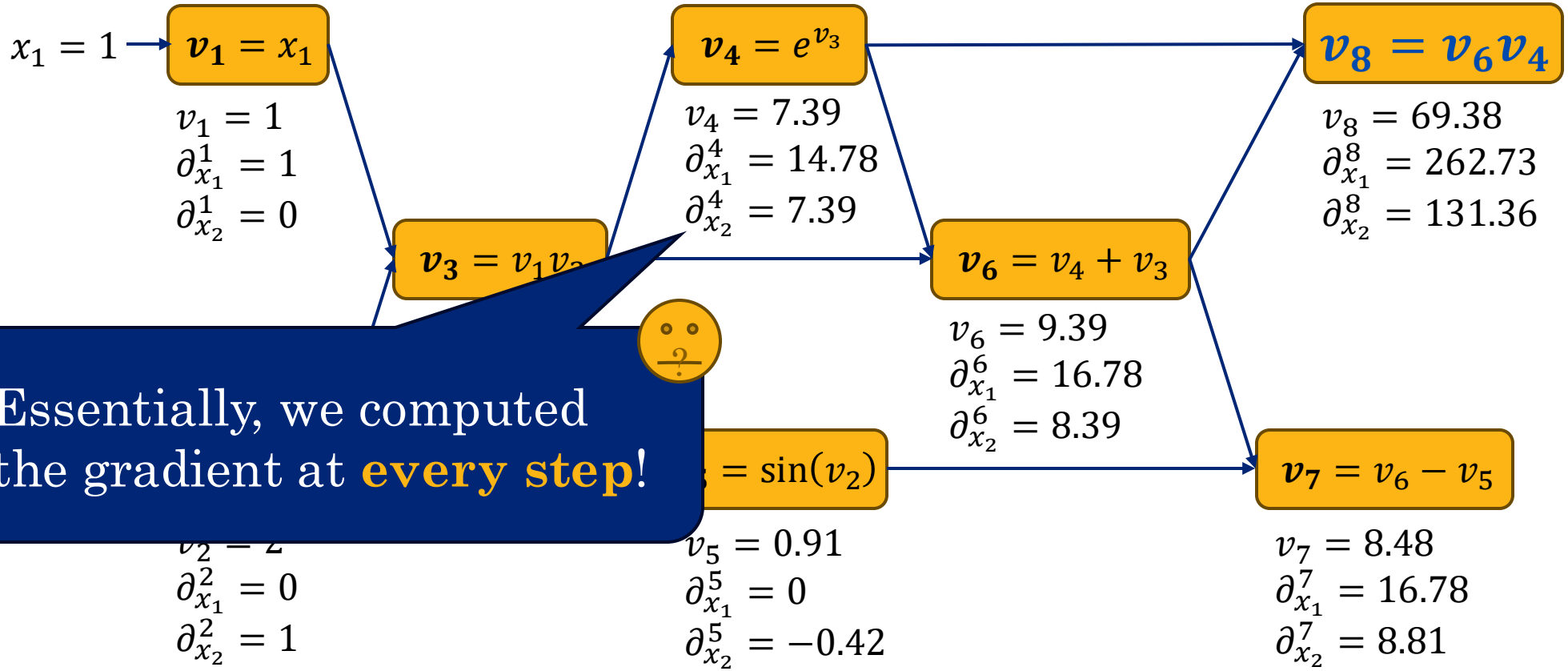


Support multiple outputs

$$v_7 = f_1(x_1, x_2) = x_1 x_2 + e^{(x_1 x_2)} - \sin(x_2)$$

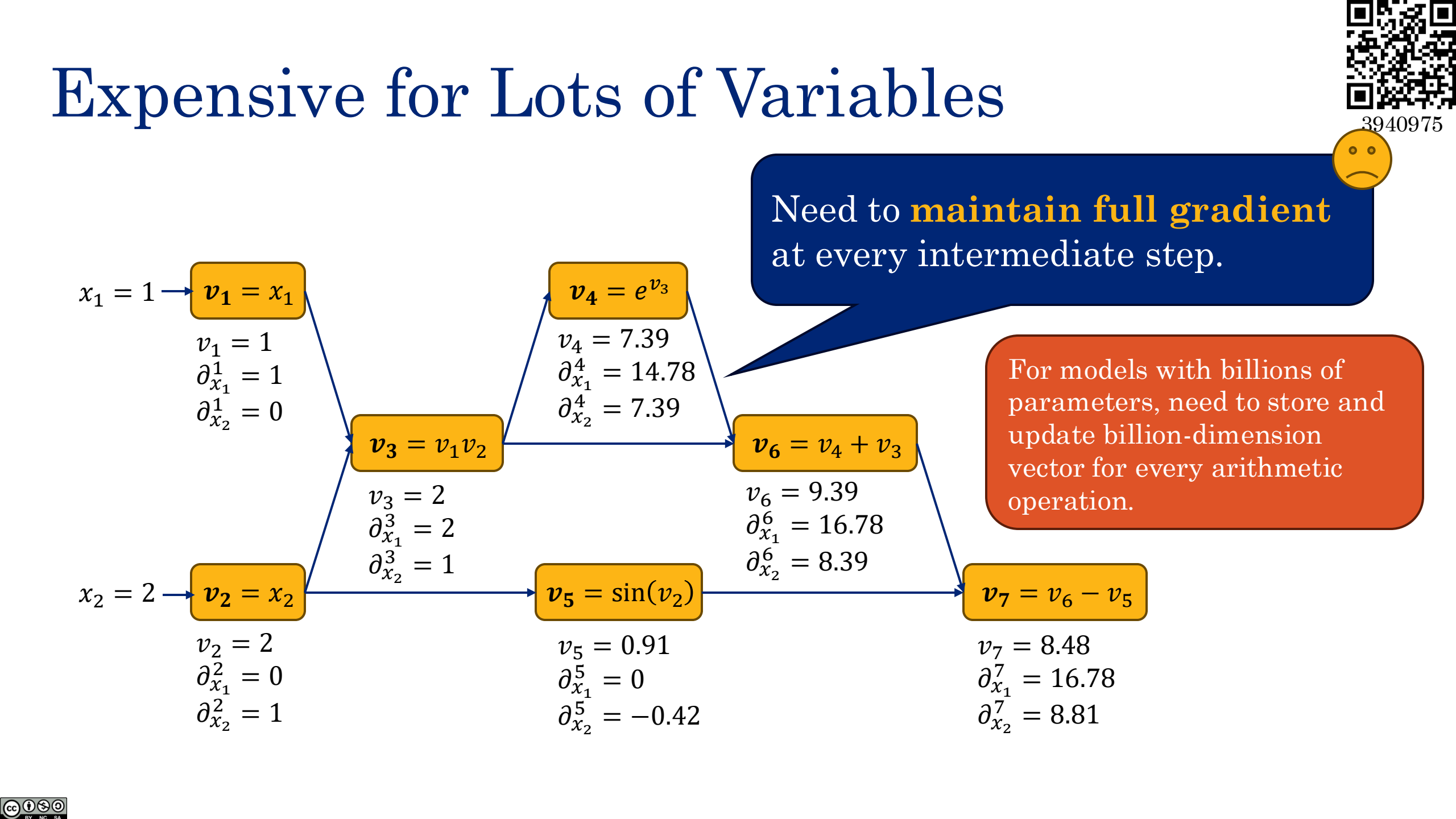
$$v_8 = f_2(x_1, x_2) = \left(x_1 x_2 + e^{(x_1 x_2)}\right) e^{(x_1 x_2)}$$

Additional function output can **reuse computation.**



Essentially, we computed the gradient at **every step**!

Expensive for Lots of Variables



Adjoint vs Tangent Variables

We have now defined **tangent** and **adjoint** **variables** for **forward** and **backward** autodiff respectively:

Tangent Variables

$$\partial_{x_k}^{i} = \sum_{j \in \mathbf{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \partial_{x_k}^{j}$$

Adjoint Variables

$$a_i^f = \sum_{j \in \mathbf{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

Notice how they build from one side of the graph to the other: